Python Beam Extraction & Monte Carlo Simulator

(PY-BEMCS)

Comprehensive Physics Reference, Numerical Controls, and User Manual

Original Author:
Dr. Bharat Singh Rawat
bharat.bharat22@gmail.com

Version 2.0 Extended Architecture & Physics Modifications:
Nick Magrin
nick.magrin@gmail.com

Version 2.0 (Extended Modular Release)

August 21, 2026

Abstract

PY-BEMCS is an open-source 2D3V (two spatial, three velocity components) electrostatic Particle-in-Cell (PIC) and Monte Carlo Collision (MCC) simulation suite engineered for ion thruster optics, neutral beam injectors (NBI), and advanced plasma extraction systems. This document provides a complete mathematical, physical, and numerical specification of the simulator, together with rigorous stability criteria, physics validation checks, and comprehensive user guidelines for the graphical interface, headless batch solvers, and automated multi-core benchmarking scripts.

Key capabilities detailed in this manual include:

- Self-consistent Bohm extraction from upstream plasma sheaths with space-charge Poisson solves and Boltzmann electron screening.
- Automatic numerical physics validation controls (Debye length resolution, plasma frequency limits, CFL velocity bounds, and dynamic domain scaling).
- 2D3V relativistic Boris particle advancement with dual Taichi GPU/CPU accelerated and pure-NumPy fallback kernels.
- Charge-Exchange (CEX) collision dynamics using Roy's neutral plume model or user-imported experimental cross-section datasets with log-log cubic spline smoothing.
- In-situ sputter erosion modeling (Matsunami yield formulation) coupled with live grid mesh boundary deletion.
- Coupled 2D thermal conduction, Stefan–Boltzmann radiative cooling, and Euler–Bernoulli cantilever thermo-mechanical grid aperture deflection.
- Modular JSON configuration architecture (`config.json`) supporting arbitrary N-grid configurations, discharge chamber plasma dynamics, and live parameter reloading.
- Automated multi-core parallel benchmarking suite for CEX erosion rates, Electron Backstreaming (EBS) limits, primary beam impingement / grid scraping, and perveance-divergence curves.
- Built-in asynchronous standalone application packager (`PyInstallerWorker`) embedded directly in the GUI.

Contents

1 Computational Framework and Architecture

1.1 Coordinate System and Discretisation

The computational domain employs a 2D Cartesian grid in (x, y), where x represents the primary axial beam extraction coordinate and y corresponds to the transverse/radial coordinate (half-aperture or axisymmetric slice). The domain extents are defined by $L_x \times L_y$ with uniform cell spacing $\Delta x \times \Delta y$:

$$n_x = \left\lfloor \frac{L_x}{\Delta x} \right\rfloor + 1, \qquad n_y = \left\lfloor \frac{L_y}{\Delta y} \right\rfloor + 1. \tag{1}$$

By default, $\Delta x = \Delta y = 0.02\,\text{mm}$ ($20\,\mu\text{m}$), ensuring the grid spacing resolves the local Debye length throughout the sheath region while preserving computational tractability.

Every macroparticle stores three-dimensional velocity components $\mathbf{v} = (v_x, v_y, v_z)$. While spatial motion is constrained to the (x, y) plane, v_z is rigorously advanced in time during magnetic rotations and collisional scattering.

1.2 Dual-Kernel Execution Engine (Taichi GPU + CPU Fallback)

To combine peak GPU performance on workstation hardware with maximum portability across containerized or headless computing clusters, PY-BEMCS implements a dual-kernel architecture:

1. **Taichi Backend (GPU/CPU)**: When Taichi is initialized, particle pushes, bilinear charge deposition, and explicit thermal conduction compile into parallel GPU kernels (CUDA, Metal, or Vulkan) running in 32-bit floating-point precision (`ti.f32`).
2. **Vectorised CPU Fallback Kernels**: If Taichi is not present (or when running inside frozen executable packages), the physics engine automatically activates vectorised CPU routines (`accumulate_rho_cpu`, `push_particles_boris_cpu`) using optimized NumPy array operations.

2 Rigorous Numerical Physics Controls and Stability Verification

To prevent unphysical numerical heating, artificial grid-aliasing instabilities, or phase-space skipping, the physics engine enforces strict automated validation checks during domain generation (`build_domain`):

2.1 Debye Length Resolution Criterion

To prevent unphysical finite-grid self-heating, the spatial cell dimensions must resolve the upstream plasma Debye length:

$$\lambda_\text{D} = \sqrt{\frac{\varepsilon_0 T_\text{e,up}}{e \cdot (0.61 n_0)}}, \tag{2}$$

where the pre-sheath factor $0.61 \approx \exp(-1/2)$ accounts for the density drop at the sheath edge. The engine strictly requires:

$$\Delta x \leq \lambda_\text{D} \quad \text{and} \quad \Delta y \leq \lambda_\text{D}. \tag{3}$$

If the cell spacing exceeds λ_D, a descriptive error is raised, prompting the user to refine $\Delta x, \Delta y$ or adjust the plasma density.

2.2 Plasma Frequency and Time-Step Integration Limits

The time step Δt must resolve both the ion plasma frequency ω_{pi} and the scaled electron plasma frequency ω_{pe}:

$$\omega_{\mathrm{pi}} = \sqrt{\frac{n_0 e^2}{m_{\mathrm{ion}} \varepsilon_0}}, \qquad \omega_{\mathrm{pe}} = \sqrt{\frac{n_0 e^2}{m_{\mathrm{e}} \varepsilon_0}}. \tag{4}$$

The maximum permissible time step is bounded by:

$$\Delta t \leq \frac{2\pi}{\omega_{\mathrm{pi}}}, \qquad \Delta t_{\mathrm{e}} \leq \frac{2\pi}{\omega_{\mathrm{pe}}}. \tag{5}$$

If Δt violates this limit, domain initialization halts to avoid numerical divergence.

2.3 CFL and Velocity Resolution Bound

Particles must not travel more than one mesh cell during a single time increment. The maximum expected particle velocity at the sheath edge includes the Bohm drift and 4 standard deviations of the thermal distribution:

$$v_{\max} = v_{\mathrm{B}} + 4 v_{\mathrm{th,i}} = \sqrt{\frac{ZeT_{\mathrm{e}}}{m_{\mathrm{ion}}}} + 4\sqrt{\frac{ZeT_{\mathrm{i}}}{m_{\mathrm{ion}}}}. \tag{6}$$

The engine enforces the Courant–Friedrichs–Lewy (CFL) velocity resolution bound:

$$\frac{\Delta x}{\Delta t} \geq v_{\max} \quad \text{and} \quad \frac{\Delta y}{\Delta t} \geq v_{\max}. \tag{7}$$

2.4 Dynamic Domain Sizing and Symmetry Boundary Conditions

The axial computational domain length L_x is dynamically calculated based on the total physical grid stack:

$$L_x = 1.5\,\mathrm{mm} + \sum_{k=1}^{N} (t_k + g_k) + 3.0\,\mathrm{mm}, \tag{8}$$

ensuring adequate space upstream for meniscus formation and downstream for plume expansion.

The transverse domain height L_y and boundary conditions adapt to the selected geometry layout:

- `half_hole` (Default): Half-pitch axisymmetric representation with specular symmetry reflection at $y = 0$ ($L_y = 0.5 r_{\mathrm{s}} + 0.30 \cdot \mathrm{pitch}$).
- `one_hole`: Full single-aperture extraction domain ($L_y = 3.0 r_{\mathrm{s}}$) with periodic boundary conditions along y.
- `two_holes`: Dual-aperture inter-hole coupling domain ($L_y = 3.0 r_{\mathrm{s}} + \mathrm{pitch}$) with periodic boundaries in y.

3 Particle-in-Cell Electrostatic Algorithm

3.1 Self-Consistent Poisson–Boltzmann Solver

The electrostatic potential $\phi(\mathbf{x})$ is governed by Poisson's equation including macroparticle ion density and thermalized Boltzmann electrons:

$$\nabla^2 \phi(\mathbf{x}) \;=\; -\frac{1}{\varepsilon_0} \left[q_{\mathrm{i}}\, n_{\mathrm{i}}(\mathbf{x}) - e\, n_{\mathrm{e}}(\mathbf{x}) \right], \tag{9}$$

where $q_\mathrm{i} = Ze$ is the ion charge, n_i is the deposited ion macroparticle number density, and the electron density n_e follows the non-linear Boltzmann relation:

$$n_\mathrm{e}(\mathbf{x}) \;=\; n_0 \exp\!\left[\frac{\min(\phi(\mathbf{x}),\, \phi_\mathrm{pl}) - \phi_\mathrm{pl}}{T_\mathrm{e}}\right], \tag{10}$$

with T_e in electron-volts (eV), n_0 the upstream plasma density, and ϕ_pl the reference plasma potential established at the upstream boundary ($x = 0$). The clipping operator $\min(\phi, \phi_\mathrm{pl})$ prevents non-physical exponential density blow-ups in regions where local potential excursions exceed the upstream plasma potential.

Equation (9) is discretised on the uniform mesh via a standard 5-point finite-difference stencil:

$$\frac{\phi_{i+1,j} - 2\phi_{i,j} + \phi_{i-1,j}}{(\Delta x)^2} + \frac{\phi_{i,j+1} - 2\phi_{i,j} + \phi_{i,j-1}}{(\Delta y)^2} = -\frac{\rho_{i,j}}{\varepsilon_0}. \tag{11}$$

The resulting sparse linear system $A\boldsymbol{\phi} = \mathbf{b}$ is factorised using SuperLU sparse LU decomposition. Non-linear electron space charge is converged via successive under-relaxation with relaxation parameter $\omega = 0.2$.

3.2 Bilinear Charge Deposition

Macroparticle charge is mapped to the four surrounding mesh nodes using bilinear interpolation weights. For a particle located at (x_p, y_p) with grid indices $i = \lfloor x_p/\Delta x \rfloor$, $j = \lfloor y_p/\Delta y \rfloor$:

$$h_x = \frac{x_p - i\Delta x}{\Delta x}, \qquad h_y = \frac{y_p - j\Delta y}{\Delta y}, \tag{12}$$

$$\rho_{i,j} \mathrel{+}= \frac{q_\mathrm{i} w}{V_\mathrm{cell}} (1 - h_x)(1 - h_y), \tag{13}$$

$$\rho_{i+1,j} \mathrel{+}= \frac{q_\mathrm{i} w}{V_\mathrm{cell}} h_x(1 - h_y), \tag{14}$$

$$\rho_{i,j+1} \mathrel{+}= \frac{q_\mathrm{i} w}{V_\mathrm{cell}} (1 - h_x)h_y, \tag{15}$$

$$\rho_{i+1,j+1} \mathrel{+}= \frac{q_\mathrm{i} w}{V_\mathrm{cell}} h_x h_y, \tag{16}$$

where $V_\mathrm{cell} = \Delta x \Delta y \Delta z$ is the cell volume with assumed unit transverse thickness $\Delta z = 1\,\mathrm{mm}$, and w is the statistical macroparticle weighting factor:

$$w = \max\!\left(\frac{n_0 V_\mathrm{cell}}{N_\mathrm{ppc}},\; 10^3\right), \tag{17}$$

with default nominal target $N_\mathrm{ppc} = 40$ particles per cell.

3.3 Electric Field Computation and Boris Pusher

Electric field components are evaluated at mesh nodes via second-order central differencing:

$$E_{x,i,j} = -\frac{\phi_{i+1,j} - \phi_{i-1,j}}{2\Delta x}, \qquad E_{y,i,j} = -\frac{\phi_{i,j+1} - \phi_{i,j-1}}{2\Delta y}, \tag{18}$$

and interpolated to particle positions. Particle trajectories are advanced using the explicit leap-frog Boris method [1]:

$$\mathbf{v}^{-} = \mathbf{v}^{n-1/2} + \frac{q\,\mathbf{E}}{2m}\,\Delta t, \tag{19}$$

$$\mathbf{t} = \frac{q\,\mathbf{B}}{2m}\,\Delta t, \qquad \mathbf{s} = \frac{2\,\mathbf{t}}{1+|\mathbf{t}|^2}, \tag{20}$$

$$\mathbf{v}' = \mathbf{v}^{-} + \mathbf{v}^{-} \times \mathbf{t}, \tag{21}$$

$$\mathbf{v}^{+} = \mathbf{v}^{-} + \mathbf{v}' \times \mathbf{s}, \tag{22}$$

$$\mathbf{v}^{n+1/2} = \mathbf{v}^{+} + \frac{q\,\mathbf{E}}{2m}\,\Delta t, \tag{23}$$

$$\mathbf{x}^{n+1} = \mathbf{x}^{n} + \mathbf{v}^{n+1/2}\,\Delta t. \tag{24}$$

This scheme guarantees exact conservation of phase-space volume and energy conservation in static magnetic fields.

4 Plasma Injection and Bohm Optics

4.1 Bohm Sheath Extraction Criterion

Ions enter the computational domain through the upstream plasma boundary ($x = 0$) with an initial axial drift satisfying the Bohm sheath velocity criterion:

$$v_{\mathrm{B}} = \sqrt{\frac{ZeT_{\mathrm{e}}}{m_{\mathrm{ion}}}}, \qquad m_{\mathrm{ion}} = M_{\mathrm{amu}} \cdot u, \tag{25}$$

where M_{amu} is the ion mass in atomic mass units and $u = 1.660\,54 \times 10^{-27}\,\mathrm{kg}$.

The injected ion current across the open sheath area A_{inj} is:

$$I_{\mathrm{ion}} = 0.61\,Zen_0 v_{\mathrm{B}} A_{\mathrm{inj}}, \tag{26}$$

where the prefactor $0.61 \approx \exp(-1/2)$ accounts for the quasi-neutral pre-sheath density drop. The number of macroparticles injected at each timestep Δt is:

$$N_{\mathrm{inj}} = \frac{I_{\mathrm{ion}}\Delta t}{Zew} + \xi_{\mathrm{stochastic}}, \tag{27}$$

with stochastic remainder accumulation ensuring exact long-term current conservation.

4.2 Thermal Velocity Sampling

Thermal spread at the sheath boundary is sampled from a 3D Maxwellian distribution at ion temperature T_{i}:

$$v_x = v_{\mathrm{B}} + \zeta_x v_{\mathrm{th,i}}, \qquad v_y = \zeta_y v_{\mathrm{th,i}}, \qquad v_z = \zeta_z v_{\mathrm{th,i}}, \tag{28}$$

where $\zeta_{x,y,z} \sim \mathcal{N}(0,1)$ are standard normal random variables, and $v_{\mathrm{th,i}} = \sqrt{ZeT_{\mathrm{i}}/m_{\mathrm{ion}}}$.

5 Multiphysics Grid Models

5.1 Arbitrary N-Grid Geometry Specification

The extraction optics consist of an arbitrary sequence of N conducting grids defined by:

- DC potential V_k (V)
- Thickness t_k (mm)

- Inter-grid axial gap to subsequent grid g_k (mm)
- Aperture radius r_k (mm)
- Upstream chamfer angle θ_k (°)

Solid boundary masks $\mathcal{M}_k(x, y)$ are dynamically synthesised and rasterised onto the finite-difference mesh.

5.2 Cantilever Thermo-Mechanical Grid Deformation

Under intense ion impingement and radiative heating, individual grid membranes experience substantial temperature rises $\Delta T_k = T_k - T_\infty$. Each grid is modeled as an annular plate segment undergoing Euler–Bernoulli cantilever bending clamped at the outer boundary ($y = L_y$) and free at the aperture edge ($y = r_k$):

$$\delta_{\max,k} = \frac{\alpha_k \Delta T_k L_k^2}{2t_k}, \qquad L_k = L_y - r_k, \tag{29}$$

where α_k is the linear thermal expansion coefficient (K^{-1}). The transverse displacement profile:

$$\delta(y) = \delta_{\max,k} \left(\frac{L_y - y}{L_k} \right)^2, \tag{30}$$

is dynamically added as an axial coordinate distortion to the grid boundaries. Whenever cumulative structural deformation exceeds 5 µm, the Poisson sparse matrix operator is automatically re-factored.

5.3 2D Thermal Diffusion and Radiative Dissipation

Thermal energy deposition from impacting ions is balanced by 2D conduction within the grid solid mask and Stefan–Boltzmann radiative cooling to space:

$$\rho_k c_{p,k} \frac{\partial T}{\partial t} = k_k \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + \dot{q}_{\mathrm{dep}} - \frac{\varepsilon_k \sigma_{\mathrm{SB}}}{\Delta z} \left(T^4 - T_\infty^4 \right), \tag{31}$$

where k_k is thermal conductivity, $c_{p,k}$ is specific heat capacity, ε_k is surface emissivity, $\sigma_{\mathrm{SB}} = 5.67 \times 10^{-8}\,\mathrm{W\,m^{-2}\,K^{-4}}$, and $T_\infty = 300\,\mathrm{K}$.

6 Collisions and Sputter Erosion

6.1 Charge-Exchange (CEX) Dynamics and Plume Neutral Model

Neutral gas density distribution throughout the grid gap and downstream region follows Roy's analytical plume expansion model [2]:

$$n_{\mathrm{n}}(x, y) = n_{\mathrm{n},0}\, a \left[1 - \frac{1}{\sqrt{1 + (r_T/R)^2}} \right] \cos\theta, \tag{32}$$

with $R = \sqrt{y^2 + (x + r_T)^2}$, $\theta = \arctan(y/(x + r_T))$, and normalisation factor $a = (1 - 1/\sqrt{2})^{-1}$.

The null-collision Monte Carlo collision (MCC) probability for an ion traversing distance $\Delta s = |\mathbf{v}|\Delta t$ is:

$$P_{\mathrm{CEX}} = 1 - \exp\left[-n_{\mathrm{n}}(x, y)\, \sigma_{\mathrm{CEX}}(E)\, |\mathbf{v}|\, \Delta t \right]. \tag{33}$$

The analytical xenon CEX cross-section formulation is:

$$\sigma_{\mathrm{CEX}}(g) = \left[-0.8821 \ln(g) + 15.1262 \right]^2 \times 10^{-20}\ \mathrm{m}^2, \tag{34}$$

where $g = |\mathbf{v}|$ is relative collision speed. If custom cross-section CSV tables are imported, this formula is dynamically superseded by cubic spline interpolation in $\log_{10}(E)$–$\log_{10}(\sigma)$ space.

6.2 Sputter Yield and Dynamic Cell Removal

Erosion of grid boundaries under ion bombardment is governed by the empirical Matsunami sputtering yield relation [3]:

$$Y(E) = \min\left[Y_0 \left(E_{\text{eV}} - E_{\text{th}}\right)^{1.5},\ 1.0\right], \tag{35}$$

where Y_0 is the sputter yield coefficient and E_{th} is the sputtering threshold energy (eV).

Cumulative damage per cell is accumulated as:

$$D_{i,j} \mathrel{+}= Y(E) \cdot \kappa_{\text{acc}}, \tag{36}$$

where κ_{acc} is the numerical time-acceleration factor. When $D_{i,j} \geq D_{\text{fail}}$, the grid cell is removed from the solid mask, exposing new interior nodes and updating the field solver in real time.

7 Modular Configuration Architecture (`config.json`)

The simulator parameters are managed through a unified, hierarchical JSON structure (`config.json`). A representative configuration is shown below:

```
{
  "beam_species": {
    "mass_amu": 131.293,
    "charge_state": 1
  },
  "grid_material": {
    "preset": "Molybdenum"
  },
  "simulation": {
    "n0_plasma": 7.73e17,
    "Te_up": 8.5,
    "Ti": 0.034,
    "Tn": 400.0,
    "n0": 5.27e18,
    "Accel": 0.5,
    "Thresh": 10000.0,
    "sim_mode": "Both",
    "geometry": "half_hole"
  },
  "discharge_chamber": {
    "radius_m": 0.02,
    "length_m": 0.10,
    "n_cusp": 4,
    "pitch_mm": 3.0
  },
  "grids": [
    { "V": 1300.0, "t": 1.2, "gap": 0.9, "r": 1.3, "cham": 20.0 },
    { "V": -150.0, "t": 0.6, "gap": 0.5, "r": 0.8, "cham": 0.0 },
    { "V": 50.0,   "t": 0.5, "gap": 4.0, "r": 1.2, "cham": 0.0 }
  ],
  "cross_sections": {
    "cx_file": "",
    "see_file": "",
    "custom_file": "",
    "spline_smoothing": 0.0
  },
  "terminal_output": {
```

```
    "grid_temperatures": true,
    "beam_divergence": true,
    "saddle_point_potential": true,
    "mean_particle_energy": true,
    "iteration_time": true
  }
}
```

Listing 1: Standard `config.json` structure

8 Graphical User Interface and Features

8.1 Menu Bar and Advanced Controls

- **Settings → Advanced Parameters...**: Configures neutralizer injection coordinates (x_n, r_n), plasma offset potential, and electron mass scaling ratio $\mathcal{R} = m_{\text{ion}}/m_e$.
- **Settings → Open Config JSON...**: Interactive file browser to load external JSON configuration profiles.
- **Settings → Reload config.json**: Hot-reloads configuration files from disk without restarting the application.
- **Settings → Build .exe...**: Compiles the current Python source code into a standalone single-file binary executable using an asynchronous background thread with live milestone tracking.
- **Beam → Ion Species...**: Configures beam ion atomic mass (amu) and ionization charge state $(+1, +2, \dots)$ with predefined presets (Xe, Kr, Ar, N_2, O_2, H_2, He, Hg, Cs).
- **Beam → Cross-Section Manager...**: Import external experimental CSV tables for CEX, SEE yield, or custom reaction channels with interactive log-log cubic spline fitting.
- **Materials → Grid Material...**: Selects material presets (Molybdenum, Steel SS316, Titanium, Graphite) or custom thermophysical properties $(k, \rho, c_p, \varepsilon, \alpha, E, Y_0, E_{\text{th}})$.

8.2 Live Diagnostics and Data Export

- **Beam Divergence Angle (θ_{95})**: Evaluated from the 95^{th} percentile of ion trajectory angles downstream of the last grid:

$$\theta_{95} = Q_{0.95}\left[\left|\arctan\left(\frac{v_y}{v_x}\right)\right|\right]. \tag{37}$$

- **Electron Backstreaming (EBS) Saddle Potential**: Continuous monitoring of centerline potential in the accelerator aperture to verify backstreaming suppression ($V_{\text{sp}} < -5\,\text{V}$).
- **Ion Energy Distribution Function (IEDF)**: Live spectral histogram tracking particle energy distributions exiting the grid system.
- **Asynchronous Data Exporters**: Background progress dialogs for exporting time-series simulation telemetry to CSV, particle phase-space archives, and animated GIF recordings.

9 Multi-Core Benchmarking Suite (`benchmarks/`)

The `benchmarks/` directory contains dedicated multi-core physics sweep scripts utilising Python's `multiprocessing` engine to execute parametric studies across independent CPU worker processes simultaneously:

Benchmark Script	Physics Objective	Generated Output
`benchmark_cex.py`	Sputter erosion rate vs background neutral density ($n_n = 10^{18}$–$5 \times 10^{19}\,m^{-3}$) across multiple grid gap configurations.	`cex_benchmark_multicore.png`
`benchmark_ebs.py`	Centerline saddle-point potential barrier vs accelerator grid voltage ($V_a = -400$–$0\,V$) evaluating the EBS safety boundary.	`ebs_benchmark_multicore.png`
`benchmark_impingement.py`	Direct primary beam ion grid interception and scraping percentage vs perveance ($P = I_{ion}/V_{tot}^{1.5}$).	`impingement_ratio_benchmark.png`
`benchmark_perveance.py`	Beam divergence angle vs perveance over 60 plasma density increments across multiple grid gap spacings.	`divergence_vs_perveance_benchma`
`benchmark_perveance_Vs_Sweep.py`	Beam divergence response as a function of screen grid extraction potential ($V_s = 600$–$2000\,V$).	`divergence_vs_voltage_benchmark`

Table 1: Summary of multi-core benchmarking scripts.

10 Standalone Executable Compilation Architecture

The embedded executable builder (`Settings` → `Build .exe...`) runs PyInstaller asynchronously via the `PyInstallerWorker(QThread)` class. The compilation process proceeds without blocking the Qt event loop:

1. Spawns a dedicated background worker thread and builds inside a temporary directory (`_exe_build_tmp/`) to keep the repository root clean.
2. Intercepts real-time standard output and maps build milestones (analysis, dependency collection, PYZ archive creation, binary packaging) to a 0–100% progress dialog.
3. Automatically copies the resulting single-file executable and accompanying `config.json` into the user-selected destination directory.
4. Cleans up intermediate build artifacts upon completion or cancellation.

11 Summary of Execution Modes

11.1 Interactive Graphical Run

```
python Python/main.py
```

11.2 Headless Parameter Sweep Execution

```
python Python/run_simulation_from_config.py
```

11.3 Parallel Multi-Core Benchmarks

```
python Python/benchmarks/benchmark_perveance.py
python Python/benchmarks/benchmark_cex.py
python Python/benchmarks/benchmark_ebs.py
```

References

[1] C. K. Birdsall and A. B. Langdon, *Plasma Physics via Computer Simulation*, Institute of Physics Publishing, Bristol, 1991.

[2] R. I. S. Roy, *Numerical Simulation of Ion Thruster Plume Backflow*, Ph.D. Thesis, Massachusetts Institute of Technology, Cambridge, MA, 1995.

[3] N. Matsunami *et al.*, "Energy dependence of the yields of ion-induced sputtering of monatomic solids," *Atomic Data and Nuclear Data Tables*, vol. 31, no. 1, pp. 1–80, 1984.